`Context` objects contain the data that is used by the template when it is rendered. They take a dictionary containing the data you want in the context as an argument. To render a template, you call `Template.render()`, which returns the resulting string.

In keeping with a grand programming tradition, your first template is "Hello, World!". Figure 3.1 illustrates the concept. A `Template` object is instantiated with a template string, `Hello {{ name }}`; then a `Context` object is created with a dictionary with the key `name`; and then the `Template` object's `.render()` method is called with the `Context` as an argument. The result is the rendered template. Figure 3.1 shows this interaction.

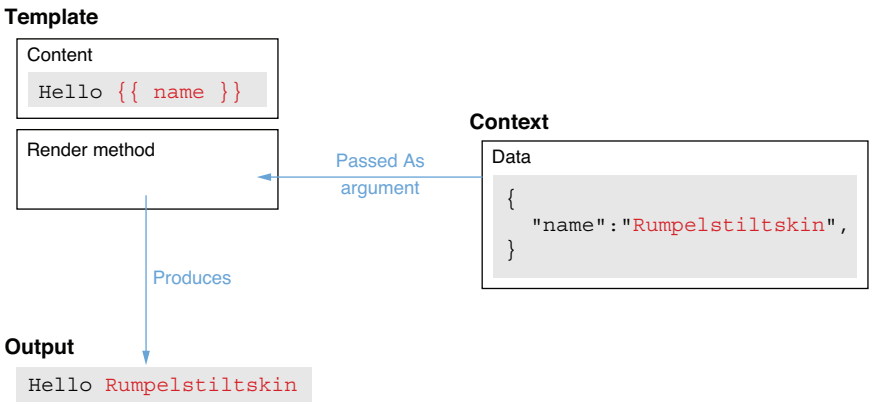


Figure 3.1 The `render()` shortcut saves you code

Double braces indicate variable replacement in a template. In this example, the `{{ name }}` causes the template engine to look for a key called `name` in the `Context`. The contents of the corresponding value are used to replace the double-brace content in the template.

Let's play with this in the Django REPL. Use the `shell` management command like you did earlier to start a new session. Once in the REPL, import the `Template` and `Context` from `django.template`. Instantiate a `Template` object, passing in `"Hello {{ name }}"` as the template to be rendered. This is the same as in figure 3.1. Now, instantiate a `Context` object, passing in a `dict` with key `name` and a corresponding value for your name. Call the `.render()` method on the `Template` object, passing in your `Context` object. This will render the template, returning the output string. The following listing shows just that.

Listing 3.1 Rendering a template in the Django REPL

```
>>> from django.template import Template, Context
>>> t = Template("Hello {{name}}")
```

A Template object with the template text Hello {{ name }}